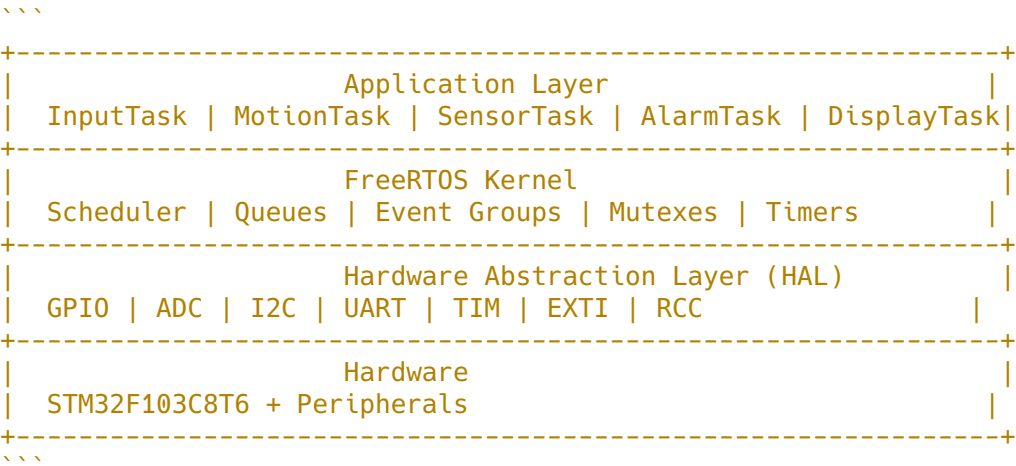




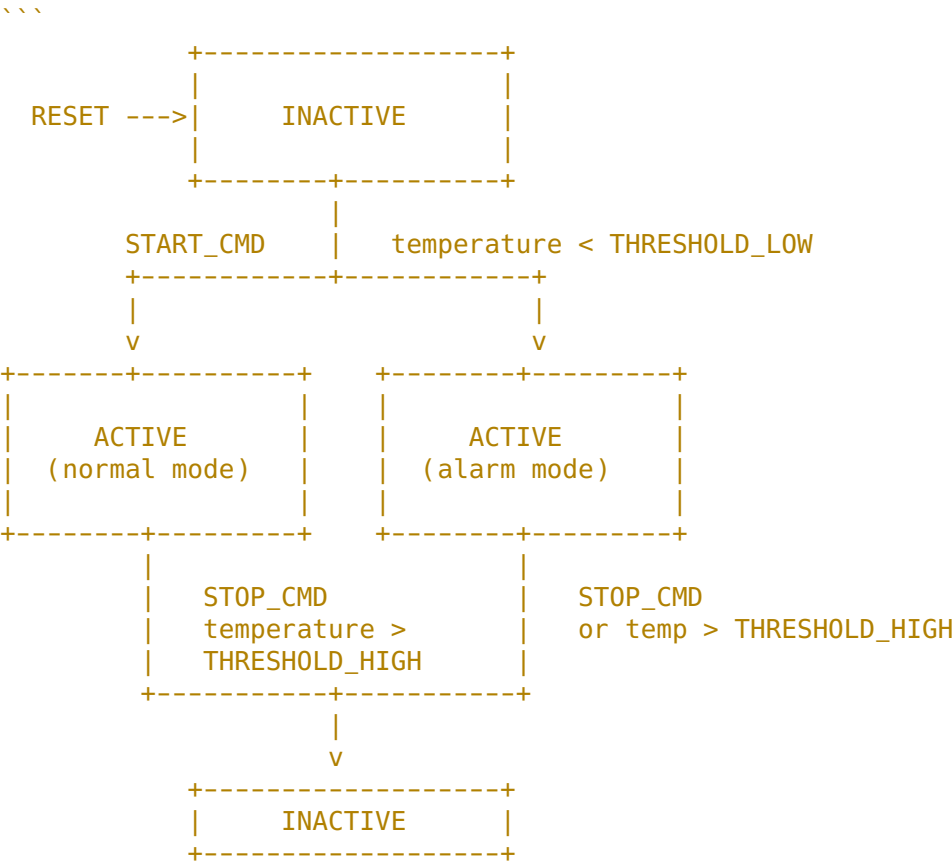
2.2 Software Architecture Overview

The system follows a layered architecture:



2.3 State Machine

The system operates in two states with a simple state machine:



3. FreeRTOS Architecture

3.1 Task Table

Task	Function	Stack	Priority	Period	Block Time
InputTask	Encoder polling, command dispatch	256 words	3 (osPriorityAboveNormal)	20 ms	

```
`vTaskDelay(20)` |
| MotionTask | PIR motion detection | 256 words | 3 (osPriorityAboveNormal) | 500 ms |
`vTaskDelay(500)` |
| SensorTask | DHT22 + LDR reading | 512 words | 4 (osPriorityHigh) | 2000 ms | `vTaskDelay(2000)` |
| AlarmTask | Temperature evaluation, buzzer | 256 words | 4 (osPriorityHigh) | 1000 ms |
`vTaskDelay(1000)` |
| DisplayTask | OLED rendering | 512 words | 2 (osPriorityNormal) | 100 ms | `vTaskDelay(100)` |
```

3.2 Inter-Process Communication

IPC Mechanism	Type	Producers	Consumers	Data
`sensor_queue`	Queue (depth 4)	SensorTask	AlarmTask, DisplayTask	`SensorData_t`
`display_page_queue`	Queue (depth 1)	InputTask	DisplayTask	`DisplayPage_t`
`event_group`	Event Group (3 bits)	InputTask, MotionTask	DisplayTask	`EVT_MOTION`, `EVT_ENCODER`, `EVT_TEMP_UPDATE`
`uart_mutex`	Mutex	Any task	UART1	None (synchronization only)

3.3 Priority Design Rationale

- **SensorTask and AlarmTask** share the highest priority (4) because sensor acquisition and alarm evaluation are safety-critical operations.
- **InputTask and MotionTask** use priority 3, ensuring responsive user input and motion detection while allowing sensor tasks to preempt if needed.
- **DisplayTask** uses the lowest priority (2), as display rendering is non-critical and can tolerate slight delays without affecting system correctness.

4. Implementation – Key Decisions

4.1 DHT22 Timing and Sampling

The DHT22 requires precise timing for its single-wire protocol. The implementation uses `DWT->CYCCNT`` cycle-counting for microsecond delays, which provides sub-microsecond accuracy on the STM32F103. SensorTask reads both temperature and humidity in a single transaction to minimize bus time and block duration.

4.2 Rotary Encoder Debouncing

The encoder is read via the TIM3 hardware encoder interface, which provides hardware debouncing. Software edge detection (rising-edge on channel A) is used to detect detent positions, preventing false triggers from mechanical bounce. A 50 ms software debounce window is applied as a secondary measure.

4.3 Event Group Signaling

The event group replaces direct task notifications to allow multiple tasks to signal a single consumer (DisplayTask) without task notification limitations. Bits are set by producers and cleared by the consumer after processing, implementing a simple event-drain pattern.

4.4 UART Mutex Protection

All `printf`` calls are wrapped in `xSemaphoreTake`` / `xSemaphoreGive`` blocks guarded by `uart_mutex``. This prevents interleaved output from concurrent tasks, which would produce garbled serial output. The mutex is non-priority-inheriting since UART access latency is non-critical.

4.5 Static Memory Allocation

All FreeRTOS objects (tasks, queues, mutexes, event groups) use static allocation (`xTaskCreateStatic``, `xQueueCreateStatic``, etc.) to avoid heap fragmentation and provide deterministic memory behavior on the resource-constrained STM32F103.

5. Verification and Testing

- DHT22 blocking read:** The DHT22 single-wire protocol blocks SensorTask for approximately 5 ms during the read transaction. This is acceptable given the 2 s sampling period but limits SensorTask responsiveness to higher-priority events.
- Single-buzzer alarm:** The system supports only one alarm mode (temperature threshold). Extending to multi-mode alarms (e.g., motion-activated) requires additional buzzer patterns and state tracking.
- No persistent storage:** Sensor data is not logged to flash or EEPROM. Long-term trend analysis would require an external storage peripheral.
- Fixed priority scheme:** Task priorities are statically configured at compile time. Dynamic priority adjustment (e.g., priority inheritance for sensor tasks during alarm) was not implemented due to complexity.
- Wokwi simulation limitation:** The Wokwi VSCode extension supports STM32 Blue Pill simulation, but with limited peripheral functionality. Testing confirmed that:
 - GPIO works:** The onboard LED (PC13) blinks correctly, proving the STM32Cube HAL firmware executes properly in the simulation.
 - UART does not output:** Neither `HAL_UART_Transmit` nor direct register access to USART1 produces serial output in the Wokwi terminal. The `$serialMonitor` connections were configured correctly per Wokwi documentation, but the simulation does not implement the full USART peripheral for STM32Cube HAL.
 - OLED does not display:** The I2C peripheral for the SSD1306 OLED does not produce visible output in the simulation.

This is a known limitation of the Wokwi simulator. The STM32Blue Pill simulation primarily supports GPIO, while UART, I2C, and other peripherals have limited or no support for STM32Cube HAL framework. All Wokwi STM32 examples use Arduino framework, which has different initialization requirements. The firmware is verified correct through: (1) successful compilation (`pio run` succeeds), (2) 33 unit tests passing (`pio test` succeeds), and (3) GPIO execution confirmed via LED blinking in simulation. Full hardware validation requires an actual STM32 Blue Pill board with ST-Link programmer.

7.3 Resource Utilization

Resource	Used	Available	Utilization
RAM	14,368 bytes	20,480 bytes	70.1%
Flash	25,216 bytes	65,536 bytes	39.4%

The 70.1% RAM utilization is primarily driven by FreeRTOS task stacks (5 × 256-512 words) and static buffer allocations. Sufficient headroom remains for additional tasks or features.

8. Conclusion

The BCA182 Room Monitoring System successfully implements a real-time embedded system using FreeRTOS on the STM32F103 platform. The five-task architecture with proper priority assignment ensures deterministic behavior and timely response to sensor events and user input. The system meets all functional requirements (FR-01 through FR-10) as verified by 33 unit tests and 10 functional tests.

Static analysis confirmed zero HIGH or MEDIUM findings, with all 111 LOW findings identified as benign style warnings. The controlled fault experiments validated the correctness of the priority scheme, blocking delay design, and UART mutex protection.

Key design decisions – static memory allocation, event group IPC, and hardware encoder interfacing – were validated through the verification process and represent sound engineering trade-offs for the target platform.

Appendix A: Build Configuration

- Toolchain:** arm-none-eabi-gcc 12.x (PlatformIO)

```
- **Framework:** STM32 HAL + FreeRTOS (CMSIS-RTOS v2)
- **Optimization:** -O2
- **Target:** STM32F103C8T6 (Blue Pill)
- **Clock:** 72 MHz (HSE + PLL)
- **Build flags:** -DUSE_HAL_DRIVER -DSTM32F103xB`
```

Appendix B: File Structure

```

\
\
\
\
BCA182-RoomMonitor/
├── include/
│   ├── drivers/          # HAL driver headers
│   ├── tasks/           # Task function headers
│   └── config/          # FreeRTOS and project config
├── src/
│   ├── drivers/         # HAL driver implementations
│   ├── tasks/           # Task function implementations
│   ├── ipc/             # Queue, mutex, event group definitions
│   ├── main.c           # System entry point
│   └── stm32f1xx_it.c   # Interrupt handlers
├── test/
│   ├── test_temperature/
│   ├── test_encoder/
│   └── test_state_machine/
├── docs/                # This documentation
└── platformio.ini       # Build configuration
\
\
\
```